Polynomial System Solving via Factor Branching

Ilay Menahem[a], Michael Shalyt[a] and Ido Kaminer[a,b]

[a]Andrew and Erna Viterbi Faculty of Electrical and Computer Engineering, Technion – Israel Institute of Technology, Haifa, 3200003, Israel
[b]Department of Materials Science and Engineering, Technion – Israel Institute of Technology, Haifa, 3200003, Israel

ARTICLE INFO

Keywords:
Polynomial systems
Gröbner bases
Factorization
Minimal hitting sets
Primary decomposition

ABSTRACT

Polynomial systems are a fundamental language for describing algebraic and geometric problems in mathematics and its applications. Beyond finding individual solutions, one often seeks global properties of the solution space, such as its dimension and irreducible components, or structural properties and decompositions of the underlying polynomial ideal. For large polynomial systems with many variables and high degrees, determining these global properties can become intractable. Leading approaches include variants of Buchberger's algorithm and resultants. However, even the most modern approaches still often fail on practical problems or slight perturbations of standard benchmarks. Here we introduce a framework that divides a polynomial system into simpler branches by interreducing and factoring its equations. The framework only assumes that the solver used on each branch describes satisfiable (SAT) branches and certifies unsatisfiable (UNSAT) ones. Its variants return the minimal primes and a minimal primary decomposition of the ideal, and we prove every variant exact. In our experiments, we use Gröbner-basis solvers. On 20 generated and 25 size-graded benchmark systems, factor branching completes 19 and 21 within 60 seconds, against at most 16 and 14 for the baselines. On a system arising from research in quantum field theory, all nine baselines time out, whereas factor branching finishes in under a second and its primary variant computes a primary decomposition in under five minutes. Together, these ideas offer a path toward the exact analysis of previously inaccessible polynomial systems.

1. Introduction

Systems of multivariate polynomial equations appear whenever a symbolic model contains nonlinear algebraic constraints. Applications include robotics, coding theory, optimization, statistics, and physics (Sturmfels, 2002). In many of these problems, an isolated solution is not enough. One needs the dimension of the solution set, its irreducible components, or the exact relations that hold on every solution. These are questions about the ideal I generated by the equations and its variety $V(I)$, rather than about a single point (Cox et al., 2015; Kreuzer and Robbiano, 2000).

The outputs of algebraic algorithms vary along two axes (Figure 1):

- How far the output breaks down the ideal: it treats I as one piece, decomposes it into ideals that need not be prime or primary, or decomposes it into its prime or primary components.
- How much information the output retains: whether $V(I)$ is empty, properties of $V(I)$ such as its dimension, $\sqrt{I}$, or I itself.

Elimination by resultants provided an early systematic way to attack polynomial systems, but the introduction of Gröbner bases gave a common algorithmic framework for elimination, ideal membership, dimension, and related tasks (Cox et al., 2015; Kreuzer and Robbiano, 2000). Buchberger developed this framework in his 1965 thesis (Buchberger, 2006). Later algorithms changed how the expensive reductions are organized. F4 replaces individual polynomial reductions with sparse linear algebra, while F5 uses signatures to avoid many reductions to zero (Faugère, 1999, 2002). Primary-decomposition algorithms use Gröbner bases and factorization to separate components (Gianni et al., 1988). Factorizing Gröbner-basis methods split on individual polynomial factorizations and preserve the solution set; these methods also interreduce and prune redundant branches (Decker et al., 2025; Gräbe, 1995). Numerical algebraic geometry takes a different route: homotopy continuation computes isolated roots numerically, while witness sets represent positive-dimensional components (Sommese et al., 2001; Sommese and Wampler, 2005).

Recent work has also explored machine-learning methods for polynomial systems. Peifer et al. (2020) trained a reinforcement-learning policy for pair selection in Buchberger's algorithm. Kera et al. (2024) trained transformers to

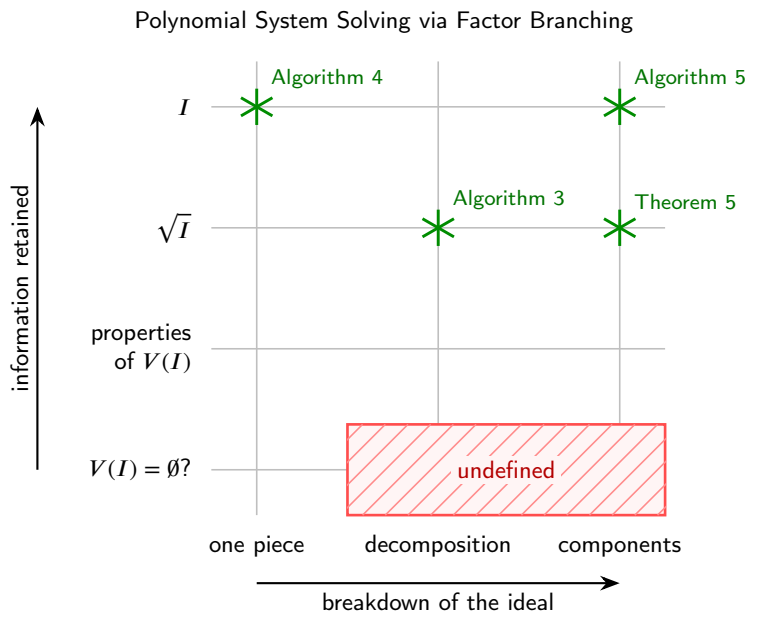


Figure 1: Outputs of polynomial-system algorithms on two axes. Columns order the outputs by how far they break down the ideal I, and rows by how much information they retain. Crosses mark the methods of this paper. The hatched cells are undefined, since whether $V(I)$ is empty is a single answer with nothing to break down.

compute Gröbner bases directly for a restricted class of zero-dimensional radical ideals. Yet these works are limited to small and specific polynomial ideals.

Despite this progress, large systems remain inaccessible, such as a quantum-gravity system from our research on which every standalone method in our comparison times out (Subsection 5.4).

Here, we introduce solver-agnostic methods for several of these outputs, all built from the same steps. Each method factors the equations and enumerates the minimal hitting sets of the resulting factor sets. A minimal hitting set contains at least one factor of every equation, so it defines a branch, a smaller system whose solutions are solutions of the original one, and $V(I)$ is the union of their solution sets. Interreducing and factoring a branch can split it further, and the methods repeat the split until it changes nothing. Each method then calls an exact solver on the branches and combines the results into its output. Unlike factorizing Gröbner-basis methods, which split on one factorization at a time inside a Gröbner-basis computation, the methods split on all equations at once before calling the solver, so any exact solver can serve. Since the worst-case complexity of computing a Gröbner basis is doubly exponential in the number of variables (Mayr and Meyer, 1982), breaking down a large system into smaller branches can be substantially faster than solving the original system directly.

Figure 1 places the methods of this paper on the two axes. The methods differ in the branch solver and in how they combine the branches. Factor branching asks the solver to describe each branch, including any positive-dimensional families, or to certify that it is UNSAT, and so returns a decomposition of $V(I)$ whose pieces may overlap (Section 3). Prime factor branching asks for the minimal primes of each branch and returns those of I, which give the irreducible components of $V(I)$ over K and their dimensions (Theorem 5). The remaining methods keep the exponents of the factors, and with them I itself. Ideal reduction uses the SAT branches to replace generators by smaller members of I (Subsection 4.1). Primary factor branching obtains the isolated primary components of I from its minimal primes and runs a general primary decomposition only when I has embedded components (Subsection 4.2).

We prove that every method is exact. Refinement terminates and preserves the solution set, ideal reduction preserves I, and prime and primary factor branching return the minimal primes and a minimal primary decomposition of I. Every numbered result is also formalized in Lean 4 with Mathlib, in the repository listed under Data availability. On generated systems with factor structure and on size-graded families from SymbolicData and from applications, factor branching with the best of its backends completes 19 of 20 and 21 of 25 systems within 60 seconds, against at most 16 and 14 for the direct Gröbner-basis and decomposition methods. On a system from quantum field theory, all nine standalone baselines time out, whereas factor branching with the `facstd` backend completes in under a second. Primary factor

branching computes the primary decomposition of this system in under five minutes, while Singular's `primdecSY` does not finish within 30 minutes on the original equations.

2. Preliminaries and notation

2.1. Polynomial systems, ideals, and varieties

Let K be a field, let $\overline{K}$ be its algebraic closure, and write $R = K[x_1, \dots, x_n]$. The input is a finite system $P = \{p_1, \dots, p_m\} \subseteq R$ of nonzero polynomials. Unless stated otherwise, a solution is a point in $\overline{K}^n$. For a finite set $Q \subseteq R$, its solution set is

$$V(Q) = \{a \in \overline{K}^n : q(a) = 0 \text{ for every } q \in Q\}.$$

We call Q satisfiable (SAT) if $V(Q) \neq \emptyset$ and unsatisfiable (UNSAT) if $V(Q) = \emptyset$.

We write $\langle Q \rangle$ for the ideal generated by Q.

For $X \subseteq \overline{K}^n$, its vanishing ideal $I(X)$ contains every polynomial in R that vanishes on all of X. The radical of an ideal J is $\sqrt{J} = \{g \in R : g^r \in J \text{ for some } r \geq 1\}$. Hilbert's Nullstellensatz gives

$$I(V(Q)) = \sqrt{\langle Q \rangle}.$$

Thus $V(Q) = \emptyset$ exactly when $\langle Q \rangle = \langle 1 \rangle$ (Cox et al., 2015; Kreuzer and Robbiano, 2000).

2.2. Saturation and decompositions

The saturation of an ideal J by $u \in R$ (Cox et al., 2015, Ch. 4, §4) is

$$J : u^\infty = \{g \in R : u^m g \in J \text{ for some } m \geq 0\}.$$

An ideal $\mathfrak{q}$ is primary if $gh \in \mathfrak{q}$ implies $g \in \mathfrak{q}$ or $h^m \in \mathfrak{q}$ for some $m \geq 1$, and then $\sqrt{\mathfrak{q}}$ is prime. Every ideal J of R has a minimal primary decomposition (Cox et al., 2015, Ch. 4, §8)

$$J = \mathfrak{q}_1 \cap \cdots \cap \mathfrak{q}_t$$

into primary ideals with distinct radicals $\mathfrak{p}_j = \sqrt{\mathfrak{q}_j}$, where no $\mathfrak{q}_j$ contains the intersection of the others. The radicals are the associated primes of J, and the minimal ones among them are the minimal primes of J. The minimal primes intersect to $\sqrt{J}$ (Cox et al., 2015, Ch. 4, §6). The components whose radicals are minimal primes, called isolated, are unique; the other components, called embedded, are not (Cox et al., 2015, Ch. 4, §8). Saturation keeps the components whose radicals do not contain u, $J : u^\infty = \bigcap_{u \notin \mathfrak{p}_j} \mathfrak{q}_j$ (Atiyah and Macdonald, 1969, Prop. 4.9). A prime that contains the intersection of finitely many ideals contains one of them (Atiyah and Macdonald, 1969, Prop. 1.11).

2.3. Monomial orders and Gröbner bases

Let $\mathbb{M}_n = \{x^\alpha : \alpha \in \mathbb{N}^n\}$ be the monomials of R. A monomial order $\prec$ is a total order on $\mathbb{M}_n$ such that every nonempty subset has a least element and

$$u \prec v \quad \Longrightarrow \quad uw \prec vw \qquad (u, v, w \in \mathbb{M}_n).$$

For a nonzero polynomial p, the leading monomial $\mathrm{LM}(p)$ is the greatest monomial of p under $\prec$, $\mathrm{LC}(p)$ is its coefficient, and $\mathrm{LT}(p)$ is its leading term.

For an ideal J, define its leading-monomial ideal by

$$\mathrm{LM}(J) = \langle \mathrm{LM}(f) : f \in J \setminus \{0\} \rangle.$$

A finite set $G \subseteq J$ is a Gröbner basis of J if

$$\mathrm{LM}(J) = \langle \mathrm{LM}(g) : g \in G \rangle.$$

A finite set Q is *interreduced* if every element is monic and no term of one element is divisible by the leading monomial of another. Algorithm 1 interreduces a finite set of nonzero polynomials.

Algorithm 1 Interreduction

Require: A finite set $Q \subseteq R \setminus \{0\}$ and a monomial order $\prec$
Ensure: An interreduced set that generates $\langle Q \rangle$

while there exist distinct $p, q \in Q$ and a term cu of p with $\mathrm{LM}(q) \mid u$ **do**
  $r \leftarrow p - \dfrac{cu}{\mathrm{LT}(q)} q$
  $Q \leftarrow Q \setminus \{p\}$
  if $r \neq 0$ **then**
    $Q \leftarrow Q \cup \{r\}$
  end if
end while
$Q \leftarrow \{p/\mathrm{LC}(p) : p \in Q\}$
return Q

The loop terminates. Let $\mu(Q)$ be the multiset of the monomials of all elements of Q. An iteration replaces the monomials of p by those of r, which lack u and are monomials of p or strictly smaller than u. If $r = 0$ or r already lies in Q, the iteration only removes the monomials of p. Either way, $\mu(Q)$ decreases in the multiset extension of $\prec$, the order generated by replacing an element of a multiset with finitely many smaller ones. It is well founded because $\prec$ is (Dershowitz and Manna, 1979). Interreduction preserves $\langle Q \rangle$ and $V(Q)$ (Cox et al., 2015, Ch. 2, §7). Since $\mathrm{LM}(r) \preceq \mathrm{LM}(p)$, it does not increase the multiset of leading monomials, and it does not shrink the ideal $M(Q) = \langle \mathrm{LM}(q) : q \in Q \rangle$: an iteration with $u \neq \mathrm{LM}(p)$ keeps $\mathrm{LM}(p)$ as the leading monomial of r, and if $u = \mathrm{LM}(p)$, then $\mathrm{LM}(q)$ divides it and q stays in Q.

If Q is interreduced and a monic g divides some $q \in Q$ with $\mathrm{LM}(g) \in M(Q)$, then $g = q$. Indeed, some $q' \in Q$ has $\mathrm{LM}(q') \mid \mathrm{LM}(g) \mid \mathrm{LM}(q)$, so $q' = q$, and q/g is a constant, equal to 1 because g and q are monic.

For a fixed monomial order, every ideal J has a unique interreduced Gröbner basis, its reduced Gröbner basis. The reduced basis is $\{1\}$ if and only if $J = \langle 1 \rangle$. Therefore, a Gröbner basis computation can describe a branch and detect when it is UNSAT (Cox et al., 2015; Kreuzer and Robbiano, 2000).

Reduction also tests containment. A polynomial g reduces to zero modulo a finite set G if repeatedly replacing g by $g - \frac{cu}{\mathrm{LT}(q)} q$, for a term cu of g and $q \in G$ with $\mathrm{LM}(q) \mid u$, reaches 0. If every element of a finite set G' reduces to zero modulo G, then $\langle G' \rangle \subseteq \langle G \rangle$, and hence $V(G) \subseteq V(G')$. If G is a Gröbner basis, the test is exact: $\langle G' \rangle \subseteq \langle G \rangle$ if and only if every element of G' reduces to zero modulo G (Cox et al., 2015, Ch. 2, §§3, 6).

2.4. Factor sets and hitting sets

Fix a factorization over K of each polynomial q of a finite system $Q \subseteq R \setminus \{0\}$:

$$q = c_q \prod_{f \in C_q} f^{e_{q,f}},$$

where $c_q \in K \setminus \{0\}$, the factor set C_q contains the distinct monic, nonconstant irreducible factors of q, and $e_{q,f} \geq 1$. We write $C(Q) = \bigcup_{q \in Q} C_q$. A point satisfies $q = 0$ exactly when at least one factor in C_q vanishes at that point.

A hitting set of Q is a set $H \subseteq C(Q)$ such that $H \cap C_q \neq \emptyset$ for every $q \in Q$. It is minimal if no proper subset is also a hitting set (Berge, 1989). We write $\mathcal{H}(Q)$ for the family of minimal hitting sets of Q. Each $H \in \mathcal{H}(Q)$ defines a branch in which all factors in H vanish.

3. Factor branching framework

In this section, we present a method that breaks a polynomial system P into simpler branches, from which a solver describes $V(P)$ and $\sqrt{\langle P \rangle}$ more easily than from P itself.

3.1. Framework description

Algorithm 2 breaks the system P into smaller branches by factoring each generator and enumerating the minimal hitting sets of the factor sets. Each hitting set generates a new branch, which we interreduce and factor again, until it reaches a fixed point. We call a branch *terminal* if one round leaves it unchanged.

Algorithm 2 Refinement

Require: A finite polynomial system P
Ensure: The terminal branches of P

$W \leftarrow \mathcal{H}(P)$
repeat
  $W_{\text{prev}} \leftarrow W, \quad W \leftarrow \emptyset$
  for all $Q \in W_{\text{prev}}$ **do**
    $Q \leftarrow \text{INTERREDUCE}(Q)$
    $W \leftarrow W \cup \mathcal{H}(Q)$
  end for
until $W = W_{\text{prev}}$
$W \leftarrow \text{PRUNE}(W)$ ▷ Optional
return W

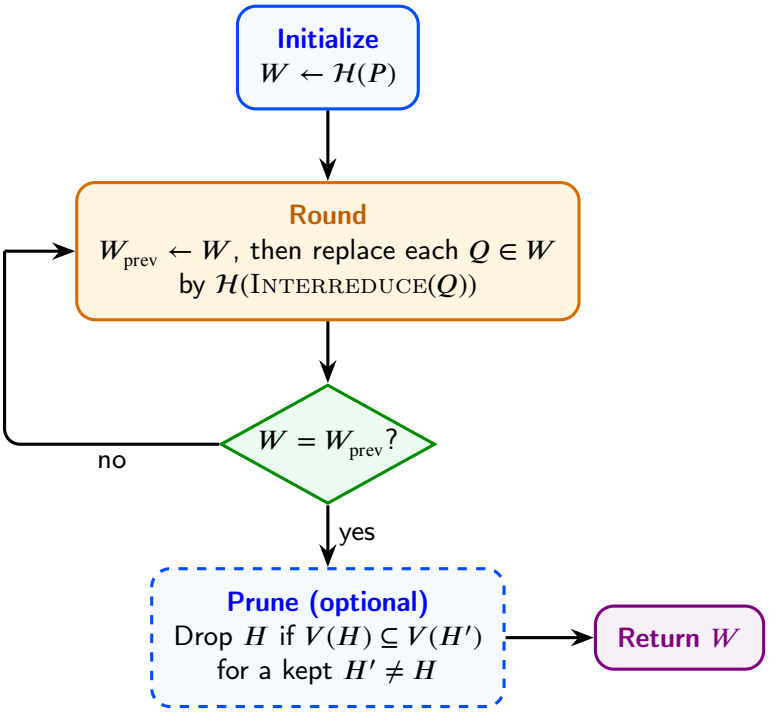


Figure 2: Refinement of Algorithm 2. Each round replaces every branch by the minimal hitting sets of its interreduction, until a round leaves W unchanged.

Hitting sets are minimal only among the children of one branch, so the loop of Algorithm 2 can end with distinct branches H and H' such that $V(H) \subseteq V(H')$. Solving H then adds no solutions. $\text{PRUNE}(W)$ removes each branch whose solution set it certifies to lie in that of another kept branch.

Interreducing P first can expose factors that its generators hide, but interreducing can also eliminate factorizations without exposing new ones. So whether to interreduce first depends on P.

Let $\mathcal{L}(P)$ denote the terminal branches returned by Algorithm 2 on P. For each $H \in \mathcal{L}(P)$, the branch solver must either describe $V(H)$ exactly or certify that $V(H) = \emptyset$. We assume that every solver in this paper terminates and returns a correct output.

A solved branch can prune the later branches whose solution sets it contains. Branches with fewer generators should then come first: they are likely to be solved faster and, being likely of higher dimension, to contain later branches.

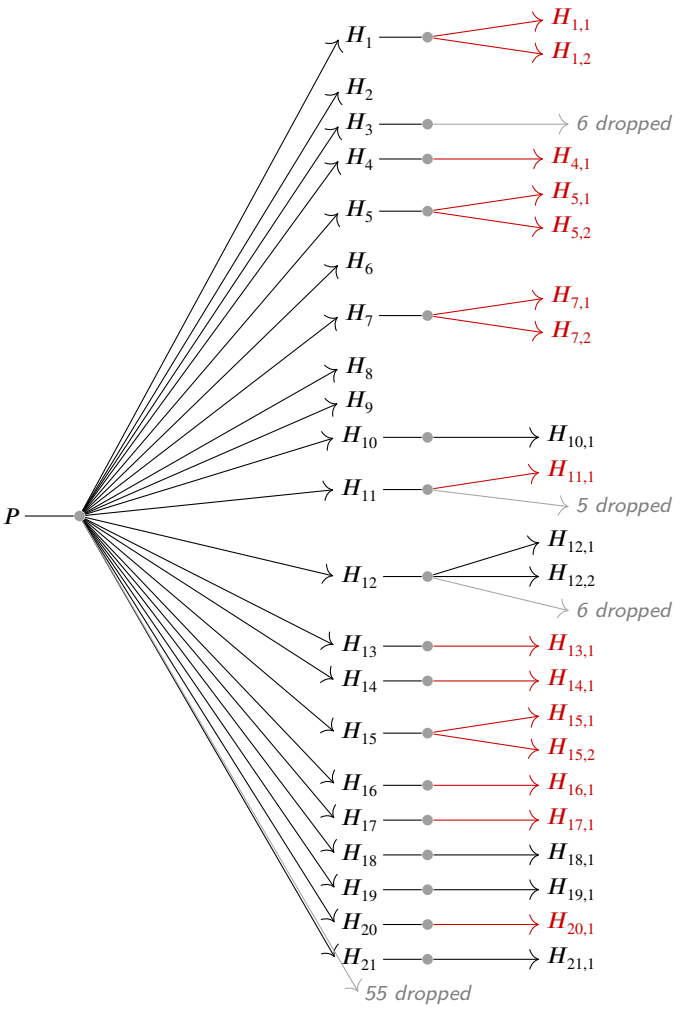


Figure 3: Refinement tree of the physical system (Section 5), as in the `facstd` row of Table 5. Each gray dot interreduces and factors a branch and branches on its minimal hitting sets, and the leaves are terminal. The red branches are the ones removed by PRUNE. Gray branches are dropped due to side conditions.

Algorithm 3 Factor branching

Require: A set of polynomials $P = \{p_1, \dots, p_m\}$ and a branch solver $\mathcal{B}$
Ensure: A decomposition of $V(P)$

$S \leftarrow \emptyset$ ▷ Solution-set representations
for all $H \in$ REFINE(P) **do**
    $S \leftarrow S \cup \mathcal{B}(H)$
end for
return S

3.2. Theoretical analysis

We show that Algorithm 3 terminates and returns a decomposition of $V(P)$ and $\sqrt{\langle P \rangle}$ by first showing that Algorithm 2 terminates and returns branches whose varieties have union $V(P)$ and whose radicals intersect to $\sqrt{\langle P \rangle}$. We then show that these branches are simpler than P.

First we need to show that the minimal hitting sets of the factor sets of a polynomial system P cover its solution set.

Lemma 1 (One-round branch decomposition). *For every finite $Q \subseteq R \setminus \{0\}$,*

$$V(Q) = \bigcup_{H \in \mathcal{H}(Q)} V(H).$$

PROOF. Let $a \in V(Q)$. Every $q \in Q$ vanishes at a, so some factor in C_q does. The factors in $C(Q)$ that vanish at a therefore form a hitting set, which contains a minimal one H, and $a \in V(H)$. Conversely, let $a \in V(H)$ for some $H \in \mathcal{H}(Q)$. Every $q \in Q$ has a factor in H, which vanishes at a, so $a \in V(Q)$.

By Lemma 1, every round of Algorithm 2 preserves the solution set, so the terminal branches describe $V(P)$ and $\sqrt{\langle P \rangle}$.

Lemma 2 (Refinement preserves solutions). *Every round of Algorithm 2 preserves $\bigcup_{Q \in W} V(Q)$. Consequently,*

$$V(P) = \bigcup_{H \in \mathcal{L}(P)} V(H) \qquad \text{and} \qquad \sqrt{\langle P \rangle} = \bigcap_{H \in \mathcal{L}(P)} \sqrt{\langle H \rangle}.$$

PROOF. Interreduction keeps $V(Q)$, so Lemma 1 gives $V(Q) = \bigcup_{H \in \mathcal{H}(\widetilde{Q})} V(H)$ for every branch Q, with $\widetilde{Q} = \textsc{Interreduce}(Q)$, and a round keeps the union. The optional PRUNE removes only branches whose solution sets lie in that of a kept branch. The loop starts from $W = \mathcal{H}(P)$, so Lemma 1 and induction on the rounds give the first identity. By the Nullstellensatz, $\sqrt{\langle P \rangle} = I(V(P))$ and $\sqrt{\langle H \rangle} = I(V(H))$, and the vanishing ideal of a finite union is the intersection of the vanishing ideals, which gives the second.

Next we show that Algorithm 2 terminates. First note that each generator of an interreduced branch is a multiple of an element of each child, so the leading monomials of a child generate a larger monomial ideal, strictly unless the child is terminal, and interreducing the child keeps this ideal from shrinking. Since R is Noetherian, these ideals cannot grow forever, so every branch eventually becomes terminal, and a round changes no terminal branch.

Theorem 1 (Termination of refinement). *Algorithm 2 terminates on every finite polynomial system.*

PROOF. One round replaces each branch Q by its children $\mathcal{H}(\widetilde{Q})$, where $\widetilde{Q} = \textsc{Interreduce}(Q)$, independently of the other branches. It leaves a set of terminal branches unchanged, and once it leaves W unchanged, so does every later round.

We first show that every child H of Q is terminal or satisfies $M(\widetilde{Q}) \subsetneq M(\widetilde{H})$. Every $q \in \widetilde{Q}$ is a multiple of some $h \in H$, so $\mathrm{LM}(h) \mid \mathrm{LM}(q)$ and $M(\widetilde{Q}) \subseteq M(H)$. If equality holds, every $h \in H$ is monic, divides some $q \in \widetilde{Q}$, and has $\mathrm{LM}(h) \in M(\widetilde{Q})$, so $h = q$ (Subsection 2.3). Hence $H \subseteq \widetilde{Q}$, and every $q \in \widetilde{Q}$ equals its factor in H, so $H = \widetilde{Q}$. Then H is interreduced and consists of monic irreducible polynomials, so $\widetilde{H} = H$ and $\mathcal{H}(H) = \{H\}$, that is, H is terminal. Otherwise $M(\widetilde{Q}) \subsetneq M(H) \subseteq M(\widetilde{H})$, since interreduction does not shrink M.

Next, we show that the rounds started from $\{Q\}$ reach a set of terminal branches. Since R is Noetherian, it has no infinite strictly ascending chain of ideals, so by Noetherian induction we may assume this for every branch H with $M(\widetilde{Q}) \subsetneq M(\widetilde{H})$. A terminal Q needs no round. Otherwise, each of the finitely many children H of Q is terminal or has $M(\widetilde{Q}) \subsetneq M(\widetilde{H})$, so its rounds reach terminal branches after some k_H rounds, and those of $\{Q\}$ after $1 + \max_H k_H$ rounds.

Hence, after some k rounds, W consists of terminal branches, and the next round leaves it unchanged, so the loop stops. Every later round leaves the returned W unchanged, so it equals the work set after round k or later, and its branches are terminal.

Algorithm 3 only calls Algorithm 2 and then $\mathcal{B}$ on each branch, so its termination and correctness follow.

Theorem 2 (Termination of factor branching). *If $\mathcal{B}$ terminates on every finite polynomial system, then Algorithm 3 terminates on every finite polynomial system.*

PROOF. By Theorem 1, Algorithm 2 terminates and returns a finite set $\mathcal{L}(P)$, so Algorithm 3 makes finitely many calls of $\mathcal{B}$, each of which terminates.

Theorem 3 (Factor branching). *Algorithm 3 terminates and returns a decomposition of $V(P)$:*

$$V(P) = \bigcup_{H \in S} V(H) \qquad \text{and} \qquad \sqrt{\langle P \rangle} = \bigcap_{H \in S} \sqrt{\langle H \rangle}.$$

PROOF. The algorithm terminates by Theorem 2, and it calls $\mathcal{B}$ on every $H \in \mathcal{L}(P)$. The branches that $\mathcal{B}$ certifies UNSAT have empty solution sets, and $\mathcal{B}$ represents every other branch exactly, so the first identity follows from Lemma 2, and the second follows from the first by the Nullstellensatz, as in the proof of Lemma 2.

Refinement is useful only if its branches are simpler than the system it starts from. Each element of a child divides a different generator of its parent, which makes the child a smaller and simpler system in the following sense.

Theorem 4 (Refined branches are simpler). *Let $H \in \mathcal{H}(\widetilde{Q})$ with $H \neq \widetilde{Q}$. The leading monomials of H are smaller than those of Q in the multiset extension of $\prec$, yet generate a strictly larger monomial ideal.*

PROOF. We first compare H with $\widetilde{Q}$, then $\widetilde{Q}$ with Q. Let $h \in H$. Since H is a minimal hitting set, $H \setminus \{h\}$ is not one, so some $q \in \widetilde{Q}$ has $C_q \cap H = \{h\}$. Let $\iota(h)$ be such a q. Then $h \mid \iota(h)$, and ι is injective, since $C_{\iota(h)} \cap H = \{h\}$ determines h. If $h \neq \iota(h)$, then $\iota(h)/h$ is not constant, since both polynomials are monic, so $\mathrm{LM}(h) \prec \mathrm{LM}(h)\,\mathrm{LM}(\iota(h)/h) = \mathrm{LM}(\iota(h))$, since $1 \prec v$ for every monomial $v \neq 1$ (Cox et al., 2015, Ch. 2, §4). The multiset of the leading monomials of H arises from that of $\widetilde{Q}$ by replacing $\mathrm{LM}(\iota(h))$ with $\mathrm{LM}(h)$ for every $h \in H$ and deleting $\mathrm{LM}(q)$ for every $q \in \widetilde{Q} \setminus \iota(H)$. Since $H \neq \widetilde{Q}$, some h has $h \neq \iota(h)$, or some q lies outside $\iota(H)$, so the multiset decreases strictly.

Interreduction does not increase this multiset and does not shrink M (Subsection 2.3). Hence the multiset of H is strictly smaller than that of Q, and $M(Q) \subseteq M(\widetilde{Q}) \subsetneq M(H)$, where the strict inclusion is shown in the proof of Theorem 1.

3.3. Minimal primes

Let $\mathcal{B}_{\min}$ be a branch solver that returns the minimal primes of its input. If we run Algorithm 3 with $\mathcal{B}_{\min}$ as a branch solver and apply the following postprocessing, we get the minimal primes of $\langle P \rangle$.

The output S is then the set of primes that $\mathcal{B}_{\min}$ returns on the branches in $\mathcal{L}(P)$, and the postprocessing keeps its minimal elements under inclusion in two steps.

1. Represent each prime by its reduced Gröbner basis.
2. Remove every prime that strictly contains another, testing containment by reduction (Subsection 2.3).

A prime that strictly contains $\mathfrak{p}'$ has smaller dimension than $\mathfrak{p}'$, so the second step tests $\mathfrak{p}' \subseteq \mathfrak{p}$ only when $\mathfrak{p}'$ has the larger dimension.

Theorem 5 (Prime factor branching). *With the branch solver $\mathcal{B}_{\min}$, the output S of Algorithm 3 after the postprocessing is the minimal primes of $\langle P \rangle$.*

PROOF. By Theorem 1, $\mathcal{L}(P)$ is finite, so S is finite. On each $H \in \mathcal{L}(P)$, $\mathcal{B}_{\min}$ returns the minimal primes of $\langle H \rangle$, which intersect to $\sqrt{\langle H \rangle}$, so Lemma 2 gives

$$\bigcap_{\mathfrak{p} \in S} \mathfrak{p} = \bigcap_{H \in \mathcal{L}(P)} \sqrt{\langle H \rangle} = \sqrt{\langle P \rangle}.$$

Hence every element of S is a prime containing $\langle P \rangle$, and every prime containing $\langle P \rangle$ contains $\sqrt{\langle P \rangle}$, hence some element of S and some minimal prime of $\langle P \rangle$ (Atiyah and Macdonald, 1969, Prop. 1.11). So a minimal prime $\mathfrak{q}$ of $\langle P \rangle$ contains an element of S and, by its minimality, equals every element of S that it contains; hence $\mathfrak{q}$ is a minimal element of S. Conversely, a minimal element of S contains a minimal prime of $\langle P \rangle$, which lies in S, so the two are equal.

4. Ideal-preserving factor branching

Algorithm 3 discards the exponents $e_{q,f}$ and hence only describes $V(P)$ and $\sqrt{\langle P \rangle}$. But ideal membership, multiplicities, and embedded components depend on $I = \langle P \rangle$ itself. This section adapts factor branching so that it preserves the ideal.

4.1. Ideal reduction

For a finite system Q and $h \in C(Q)$, let $a_h = \max\{e_{q,h} : q \in Q,\ h \in C_q\}$ be the largest exponent of h.

Suppose that a factor h lies in every SAT minimal hitting set of Q. Then h vanishes on all of $V(Q)$, and the Nullstellensatz puts some power of h in $\langle Q \rangle$. Lemma 3 shows that the power h^{a_h} suffices, so the factorization bounds the exponent without a Gröbner basis of $\langle Q \rangle$.

For example, $Q = \{xy,\ xz,\ yz - 1\}$ is interreduced for every monomial order. Its factor sets $\{x, y\}$, $\{x, z\}$, and $\{yz - 1\}$ have the minimal hitting sets $\{x, yz - 1\}$ and $\{y, z, yz - 1\}$. The second is UNSAT, since $y = z = 0$ contradicts $yz - 1 = 0$. Hence x and $yz - 1$ are in every SAT minimal hitting set, $a_x = 1$ and $a_{yz-1} = 1$, and indeed $x^1, (yz - 1)^1 \in \langle Q \rangle$.

Algorithm 4 adds each h^{a_h} to the generators, drops the generators they divide, and repeats on the new system. On the example, the first round replaces xy and xz by x, giving $\{x, yz - 1\}$. We say that $\mathcal{B}$ certifies $V(H) = \emptyset$ when Algorithm 3 with $\mathcal{B}$ certifies every terminal branch of H as UNSAT. If no minimal hitting set is SAT, the intersection is $C(Q)$, and Lemma 3 still applies.

Algorithm 4 Ideal reduction

Require: A finite polynomial system P and a branch solver $\mathcal{B}$
Ensure: A finite generating set Q of $\langle P \rangle$

```
Q ← INTERREDUCE(P)
loop
    H_sat ← {H ∈ H(Q) : B does not certify V(H) = ∅}
    Δ ← {h^{a_h} : h ∈ ⋂ H_sat} \ Q
    if Δ = ∅ then
        return Q
    end if
    Q ← INTERREDUCE(Δ ∪ {q ∈ Q : no element of Δ divides q})
end loop
```

The output generates $\langle P \rangle$, and each round replaces generators by powers of their factors, which makes the system simpler (Theorem 8).

Each round needs only the intersection of the SAT minimal hitting sets, so an implementation need not classify every minimal hitting set. It keeps the factors F that lie in every SAT hitting set checked so far. Starting from $F = C(Q)$, it skips every hitting set that contains F, intersects F with every other hitting set that is SAT, and stops once F is empty. Each skipped hitting set contains F, so the final F is $\bigcap \mathcal{H}_{\text{sat}}$. Two disjoint SAT hitting sets already empty F, so a round that finds nothing often costs only a few solver calls.

4.2. Primary factor branching

Algorithm 3 with $\mathcal{B}_{\min}$ describes only $\sqrt{I}$. A minimal primary decomposition of I also records the multiplicity of each component and embedded components. Algorithm 5 computes one with an algorithm $\mathcal{B}_{\text{prim}}$ that returns a minimal primary decomposition of its input. Its other steps make $\mathcal{B}_{\text{prim}}$ cheaper, skip it, or label its output.

The algorithm first replaces P by the output Q of Algorithm 4. By Theorem 7, $\langle Q \rangle = I$, so every later step works on a smaller description of I.

It then computes the minimal primes $\mathfrak{p}_1, \ldots, \mathfrak{p}_r$ of I, which give the isolated components in the optional lines and mark the components of $\mathcal{B}_{\text{prim}}$. Any algorithm $\mathcal{B}_{\min}$ that returns the minimal primes of its input works here, and we use prime factor branching (Theorem 5) in the experiments (Section 5). It finds the minimal primes without decomposing I itself, and it can skip the minimal hitting sets of Q that the last round of Algorithm 4 certified as UNSAT.

Optionally, the minimal primes give the isolated components $\mathfrak{q}_1, \ldots, \mathfrak{q}_r$ without $\mathcal{B}_{\text{prim}}$. They intersect to I exactly when I has no embedded components, and the algorithm then returns them, so these lines pay off only on such ideals. Each $\mathfrak{q}_i$ comes from a *separator* of $\mathfrak{p}_i$, a polynomial $w_i \in \bigcap_{j \neq i} \mathfrak{p}_j \setminus \mathfrak{p}_i$, which exists because $\mathfrak{p}_i$ contains no other minimal prime and hence, by Subsection 2.2, not their intersection. Saturating by w_i removes every component whose radical does not contain $\mathfrak{p}_i$ (Shimoyama and Yokoyama, 1996), so $\mathfrak{q}_i$ is the top-dimensional part of $I : w_i^{\infty}$, the intersection of its primary components of maximal dimension (Eisenbud et al., 1992) (Lemma 4). A Gröbner basis of I decides whether $\mathfrak{q}_1 \cap \cdots \cap \mathfrak{q}_r = I$.

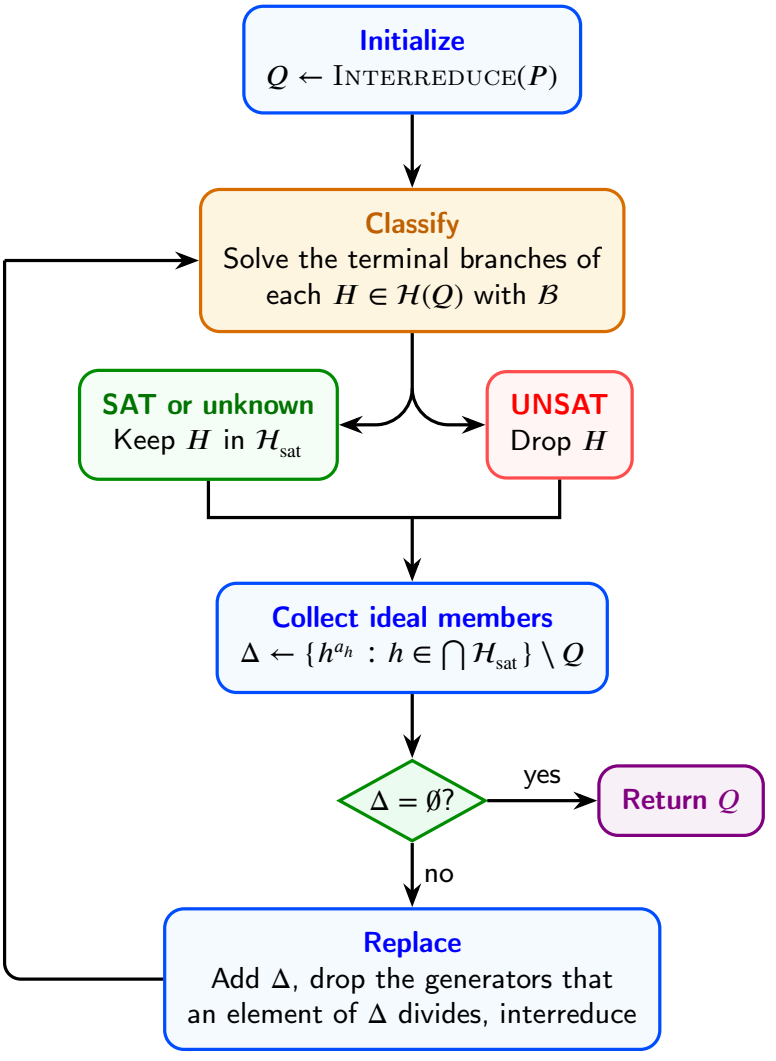


Figure 4: Ideal reduction of Algorithm 4. Every factor h chosen by all SAT minimal hitting sets enters the system as h^{a_h}, which replaces the generators it divides. The loop stops when no new power appears, and $\langle Q \rangle = \langle P \rangle$ throughout.

Otherwise, $\mathcal{B}_{\text{prim}}$ decomposes $\langle Q \rangle$, which is I.

A component $\mathfrak{q}$ is isolated if $\sqrt{\mathfrak{q}}$ is one of the $\mathfrak{p}_i$ and embedded otherwise. The radical of an embedded component strictly contains at least one $\mathfrak{p}_i$, so the variety of the component lies inside an irreducible component of $V(I)$.

For example, $I = \langle x^2, xy \rangle$ has the single minimal prime $\langle x \rangle$, which needs no separator. The top-dimensional part of I is $\langle x \rangle \neq I$, so $\mathcal{B}_{\text{prim}}$ runs and returns $I = \langle x \rangle \cap \langle x^2, y \rangle$. The component $\langle x \rangle$ is isolated, and $\langle x^2, y \rangle$ is embedded, since its radical $\langle x, y \rangle$ strictly contains $\langle x \rangle$.

A separator also needs no intersection of primes. The saturation $I : \mathfrak{p}_i^{\infty} = \{g \in R : \mathfrak{p}_i^m g \subseteq I \text{ for some } m \geq 0\}$ keeps exactly the components whose radicals do not contain $\mathfrak{p}_i$, so it lies in every other minimal prime but not in $\mathfrak{p}_i$, and every element of its Gröbner basis outside $\mathfrak{p}_i$ is a separator.

A zero-dimensional I needs neither separators nor the intersection test: its primes are maximal, so it has no embedded components. Its components are pairwise comaximal, so $\dim_K R/I$ is the sum of the $\dim_K R/\mathfrak{q}_i$, each at least $\dim_K R/\mathfrak{p}_i$ with equality only if $\mathfrak{q}_i = \mathfrak{p}_i$. If $\dim_K R/I = \sum_i \dim_K R/\mathfrak{p}_i$, every component is prime. Otherwise $\mathfrak{q}_i = I + \mathfrak{p}_i^m$ for the first m at which $\dim_K R/(I + \mathfrak{p}_i^m)$ stops growing, by Nakayama's lemma (Atiyah and Macdonald, 1969, Prop. 2.6).

Algorithm 5 Primary factor branching

Require: A finite polynomial system P, branch solvers $\mathcal{B}$ and $\mathcal{B}_{\min}$, and a minimal primary decomposition algorithm $\mathcal{B}_{\text{prim}}$
Ensure: A minimal primary decomposition of $\langle P\rangle$, each component marked isolated or embedded

$Q \leftarrow \textsc{IdealReduce}(P, \mathcal{B})$ ▷ Algorithm 4, $\langle Q\rangle = \langle P\rangle$
$\{\mathfrak{p}_1, \dots, \mathfrak{p}_r\} \leftarrow \mathcal{B}_{\min}(Q)$
for all $i \in \{1, \dots, r\}$ **do** ▷ Optional
  $w_i \leftarrow$ a separator of $\mathfrak{p}_i$
  $\mathfrak{q}_i \leftarrow$ the top-dimensional part of $\langle Q\rangle : w_i^{\infty}$
end for
if $\mathfrak{q}_1 \cap \cdots \cap \mathfrak{q}_r = \langle Q\rangle$ **then**
  return $\{\mathfrak{q}_1, \dots, \mathfrak{q}_r\}$, all marked isolated
end if
$\mathcal{D} \leftarrow \mathcal{B}_{\text{prim}}(Q)$
for all $\mathfrak{q} \in \mathcal{D}$ **do**
  mark $\mathfrak{q}$ isolated if $\sqrt{\mathfrak{q}} \in \{\mathfrak{p}_1, \dots, \mathfrak{p}_r\}$, and embedded otherwise
end for
return $\mathcal{D}$

The mark needs no equality test of ideals. A prime that strictly contains $\mathfrak{p}_i$ has smaller dimension (Subsection 3.3), so $\mathfrak{q}$ is isolated exactly when the generators of some $\mathfrak{p}_i$ of its dimension reduce to zero modulo a Gröbner basis of $\sqrt{\mathfrak{q}}$.

4.3. Theoretical analysis

We need to show that Algorithm 5 terminates and returns a minimal primary decomposition of $\langle P\rangle$ with each component marked correctly. This will be done by first showing that Algorithm 4 terminates and keeps $\langle P\rangle$, and then that the separators give the isolated components.

Lemma 3 justifies the powers that Algorithm 4 adds: each lies in the ideal. Its proof shows that the cofactors of h have no common zero on $V(Q)$ and then applies the Nullstellensatz.

Lemma 3 (SAT branches). *Let $\mathcal{H}_{\text{sat}} \subseteq \mathcal{H}(Q)$ contain every SAT minimal hitting set, and let $h \in C(Q)$ lie in every $H \in \mathcal{H}_{\text{sat}}$. Then $h^{a_h} \in \langle Q\rangle$.*

Proof. For $q \in Q$ with $h \in C_q$, let $r_q = q/h^{e_{q,h}}$ be the cofactor of h. We first show that the cofactors have no common zero on $V(Q)$. Suppose that $b \in V(Q)$ is a common zero of the cofactors. Then every $q \in Q$ has a factor $f \neq h$ with $f(b) = 0$: a factor of r_q if $h \in C_q$, and any factor that vanishes at b otherwise. So the factors $f \neq h$ in $C(Q)$ that vanish at b form a hitting set, and a minimal hitting set H inside it has $b \in V(H)$ and $h \notin H$. Then H is SAT, hence $H \in \mathcal{H}_{\text{sat}}$ and $h \in H$, a contradiction. By the Nullstellensatz, $1 = \sum_{q\in Q} g_q\, q + \sum_{q:\, h\in C_q} g'_q\, r_q$ for some $g_q, g'_q \in R$. Since $e_{q,h} \le a_h$, every $h^{a_h} r_q = h^{a_h - e_{q,h}} q$ lies in $\langle Q\rangle$, and multiplying by h^{a_h} gives $h^{a_h} \in \langle Q\rangle$.

Algorithm 4 terminates for the same reason as Algorithm 2 (Theorem 1): $M(Q)$ grows strictly in every round that does not return, since every new power has a leading monomial outside $M(Q)$.

Theorem 6 (Termination of ideal reduction). *If $\mathcal{B}$ terminates on every finite polynomial system, then Algorithm 4 terminates on every finite polynomial system.*

Proof. Consider a round with $\Delta \neq \emptyset$, and let $Q_0 = \Delta \cup \{q \in Q :$ no element of Δ divides $q\}$, so that the round ends with $\textsc{Interreduce}(Q_0)$. Every $q \in Q$ is kept or is a multiple of an element of Δ, so $M(Q) \subseteq M(Q_0)$. The inclusion is strict. Indeed, let $\delta = h^{a_h} \in \Delta$, and let $q \in Q$ have $e_{q,h} = a_h$, so that the monic δ divides q. Since Q is interreduced at the start of every round and $\delta \notin Q$, Subsection 2.3 gives $\mathrm{LM}(\delta) \notin M(Q)$. Since interreduction does not shrink M, $M(Q)$ grows strictly in every round that does not return. Since R is Noetherian, there are finitely many rounds. Each round runs Algorithm 3 with $\mathcal{B}$ on the finitely many $H \in \mathcal{H}(Q)$, which terminates by Theorem 2.

Using Lemma 3, we show that Algorithm 4 keeps the ideal.

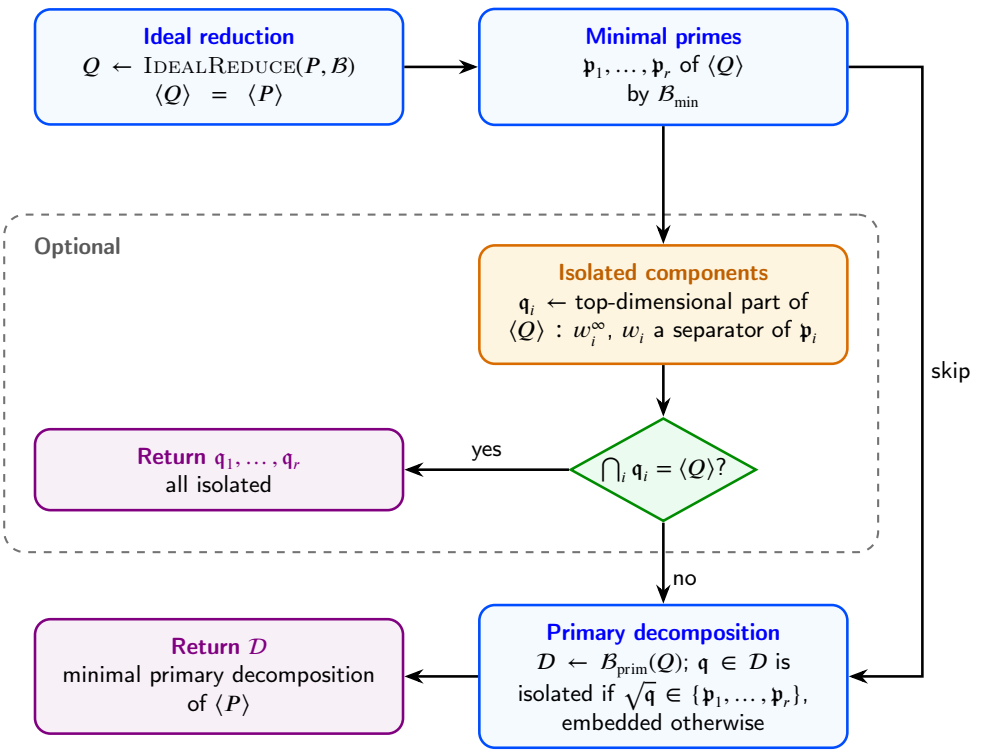


Figure 5: Primary factor branching of Algorithm 5. Ideal reduction shrinks the system to Q without changing the ideal. The separators of the minimal primes give the isolated components, which are returned if they intersect to $\langle Q \rangle$. Otherwise the general solver $\mathcal{B}_{\text{prim}}$ decomposes Q, and the minimal primes mark each component isolated or embedded.

Theorem 7 (Ideal reduction). *Algorithm 4 terminates and returns a finite generating set Q of $\langle P \rangle$.*

PROOF. The algorithm terminates by Theorem 6. Every round keeps $\langle Q \rangle$. Indeed, $\mathcal{B}$ certifies $V(H) = \emptyset$ only when it holds (Lemma 2), so $\mathcal{H}_{\text{sat}}$ contains every SAT minimal hitting set. Hence $\Delta \subseteq \langle Q \rangle$ by Lemma 3, the dropped generators are multiples of elements of Δ, and interreduction keeps the ideal.

Like the branches of Algorithm 2, the output of Algorithm 4 should be simpler than P. Each new power divides the generators it replaces, which makes every round simpler in the sense of Theorem 4.

Theorem 8 (Reduced systems are simpler). *Let a round of Algorithm 4 with $\Delta \neq \emptyset$ replace Q by Q'. The leading monomials of Q' are smaller than those of Q in the multiset extension of $\prec$, yet generate a strictly larger monomial ideal. The same holds for the output and P, unless the output is* $\textsc{Interreduce}(P)$.

PROOF. Let Q_0 be as in the proof of Theorem 6, so that $Q' = \textsc{Interreduce}(Q_0)$, and let D be the set of $q \in Q$ that some element of Δ divides. Then $Q_0 = \Delta \cup (Q \setminus D)$, and $\Delta \cap Q = \emptyset$. Let $\delta = h^{a_h} \in \Delta$, and let $q \in Q$ have $e_{q,h} = a_h$. Then $\delta \mid q$, so $q \in D$, and $\delta \neq q$, since $\delta \notin Q$. Both polynomials are monic, so q/δ is not constant, and $\mathrm{LM}(\delta) \prec \mathrm{LM}(q)$ as in the proof of Theorem 4. Assign each $\delta \in \Delta$ to one such q. The multiset of the leading monomials of Q_0 arises from that of Q by replacing $\mathrm{LM}(q)$, for every $q \in D$, with the leading monomials of the elements of Δ assigned to q, which are finitely many and smaller. Since $\Delta \neq \emptyset$, D is nonempty, so the multiset decreases strictly.

Interreduction does not increase this multiset and does not shrink M (Subsection 2.3). Hence the multiset of Q' is strictly smaller than that of Q, and $M(Q) \subsetneq M(Q')$ by the proof of Theorem 6. The loop starts from $\textsc{Interreduce}(P)$, whose multiset is at most that of P and whose M contains $M(P)$, and it changes Q only in rounds with $\Delta \neq \emptyset$. If the output is not $\textsc{Interreduce}(P)$, at least one such round runs, which gives the second claim.

The optional lines of Algorithm 5 rest on the following lemma. Saturating by a separator of $\mathfrak{p}$ keeps only components whose radicals contain $\mathfrak{p}$, and among them the $\mathfrak{p}$-primary one is the only one of maximal dimension.

Lemma 4 (Isolated components by separators). *Let $\mathfrak{p}$ be a minimal prime of an ideal I and w a separator of $\mathfrak{p}$. Then $\mathfrak{p}$ is the only minimal prime of $I : w^\infty$, and the $\mathfrak{p}$-primary component of I is the intersection of the primary components of $I : w^\infty$ of maximal dimension.*

PROOF. Let $I = \mathfrak{q}_1 \cap \cdots \cap \mathfrak{q}_t$ be a minimal primary decomposition with $\mathfrak{p}_j = \sqrt{\mathfrak{q}_j}$, and let $T = \{j : w \notin \mathfrak{p}_j\}$. By Subsection 2.2, $I : w^\infty = \bigcap_{j \in T} \mathfrak{q}_j$, and this decomposition is again minimal, since a $\mathfrak{q}_j$ that contained the intersection of the other kept components would contain the smaller intersection of all other components. For $j \in T$, the prime $\mathfrak{p}_j$ contains I, hence $\sqrt{I}$, and hence a minimal prime of I (Atiyah and Macdonald, 1969, Prop. 1.11). This minimal prime does not contain w, so it is $\mathfrak{p}$. The $\mathfrak{p}$-primary component $\mathfrak{q}$ of I is kept, since $w \notin \mathfrak{p}$. So $\mathfrak{p}$ is the only minimal prime of $I : w^\infty$, and every other kept $\mathfrak{p}_j$ strictly contains $\mathfrak{p}$ and therefore has smaller dimension (Subsection 3.3). Every minimal primary decomposition of $I : w^\infty$ has these radicals and the isolated component $\mathfrak{q}$, so $\mathfrak{q}$ is its only component of maximal dimension.

Termination of Algorithm 5 follows from that of its steps.

Theorem 9 (Termination of primary factor branching). *If $\mathcal{B}$, $\mathcal{B}_{\min}$, and $\mathcal{B}_{\text{prim}}$ terminate on every finite polynomial system, then Algorithm 5 terminates on every finite polynomial system.*

PROOF. By Theorem 6, Algorithm 4 terminates. The remaining lines make finitely many solver calls, each of which terminates.

Finally, the previous results give the correctness of Algorithm 5. It remains to show that the components the optional lines return are not redundant.

Theorem 10 (Primary factor branching). *Algorithm 5, with or without its optional lines, terminates and returns a minimal primary decomposition $\mathcal{D}$ of $\langle P \rangle$:*

$$\langle P \rangle = \bigcap_{\mathfrak{q} \in D} \mathfrak{q}.$$

It marks correctly each component isolated or embedded.

PROOF. The algorithm terminates by Theorem 9. By Theorem 7, $\langle Q \rangle = \langle P \rangle = I$, so the $\mathfrak{p}_i$ that $\mathcal{B}_{\min}$ returns on Q are the minimal primes of I.

Suppose that the optional lines return. By Lemma 4, each $\mathfrak{q}_i$ is the isolated $\mathfrak{p}_i$-primary component of I, and $\mathfrak{q}_1 \cap \cdots \cap \mathfrak{q}_r = I$. The radicals $\mathfrak{p}_i$ are distinct, and no $\mathfrak{q}_i$ is redundant. Otherwise $\mathfrak{p}_i$ contains $\bigcap_{j \neq i} \mathfrak{q}_j$, hence some $\mathfrak{q}_j$ with $j \neq i$ (Atiyah and Macdonald, 1969, Prop. 1.11), and hence $\mathfrak{p}_j$, which contradicts the minimality of $\mathfrak{p}_i$. So the $\mathfrak{q}_i$ form a minimal primary decomposition of I, and all of them are isolated.

Otherwise, $\mathcal{B}_{\text{prim}}$ decomposes $\langle Q \rangle = I$. A component is isolated exactly when its radical is a minimal prime of I, that is, one of the $\mathfrak{p}_i$, which is the mark.

In both cases, $g \in \langle P \rangle$ if and only if g lies in every component, and a Gröbner basis decides membership by reduction (Subsection 2.3).

5. Experiments

5.1. Setup

We implemented Algorithm 3 in Rust 1.98.1 with Symbolica 3.0.1 (Ruijl, 2026). Refinement prunes as in Subsection 3.1, and the branch solver computes a Gröbner basis of each terminal branch, fewest generators first, with M4GB, F5, GVW, F4, or Singular 4.4.1 (Decker et al., 2025; Faugère, 1999, 2002; Gao et al., 2016; Makarim and Stevens, 2017); a basis containing 1 certifies UNSAT. The baselines are six whole-ideal Gröbner-basis configurations and three decomposition methods from Singular and Macaulay2 1.26.06 (Decker et al., 2025; Grayson and Stillman, 2026). Unless stated otherwise, every entry is one run on an Apple M4 Pro (14 cores, 24 GB) with a 60-second limit, and TO marks a timeout. Factor branching times are wall times. The measurements are in the `results` directory of the repository listed under Data availability.

5.2. Baseline systems

The generated systems vary how many branches a system has and how hard they are. The benchmark has 20 systems in six families (Table 1). Signed-distance cycles and affine-factor hypergraphs have many cheap branches. Overlapping random multiples share dense quadratic factors across equations. Gated systems multiply every equation of Cyclic, Katsura, or a dense quadratic system by a new variable (Björck and Fröberg, 1991; Böge et al., 1986), so they have one easy selector branch and one nonlinear residual branch.

Family	Tested sizes	Variables/equations; hitting sets
Signed-distance cycle	$n = 12, 14, 16$	$12/12, 14/14, 16/16$; $924, 3432, 12870$
Overlapping random multiples	$(n, p, k) = (4, 6, 2), (6, 8, 3), (8, 10, 3)$	n/n; $4, 16, 28$
Positive affine hypergraph	$(n, m, k) = (8, 6, 4), (10, 8, 5), (12, 10, 6)$	n/m; $63, 239, 833$
Gated Cyclic	orders $5, 6, 7$	$6/5, 7/6, 8/7$; 2 each
Gated Katsura and dense quadratics	Katsura $6, 7, 8$; dense $n = 5, 6, 7$	$8/7, 9/8, 10/9, 6/5, 7/6, 8/7$; 2 each
Square affine hypergraph	$(n, k) = (8, 4), (10, 5)$	n/n; $178, 309$

Table 1
The generated systems used for the baseline benchmark.

Factor branching completes 19 of the 20 systems and is fastest on 16; the direct and decomposition classes complete 9 and 16 (Table 2). The direct methods win only on the signed-distance cycles with $n \leq 14$ and on gated Cyclic-5.

System	Size	Best factor branching	Best direct GB	Best decomposition
Signed distance	12	F5 2.669	**F4 0.443**	primdec 3.056
	14	GVW 9.824	**Singular 7.421**	primdec 29.894
	16	**M4GB 37.391**	TO	TO
Overlapping	(4, 6, 2)	**F5 0.014**	F4 0.429	facstd 0.265
	(6, 8, 3)	**F5 0.160**	TO	TO
	(8, 10, 3)	**Singular 2.671**	TO	TO
Positive affine	(8, 6, 4)	**M4GB 0.180**	TO	facstd 0.368
	(10, 8, 5)	**F4 0.703**	TO	facstd 1.836
	(12, 10, 6)	**F4 2.398**	TO	facstd 53.318
Gated Cyclic	5	F4 0.011	**GVW 0.009**	facstd 0.181
	6	**Singular 0.043**	Singular 0.309	M2 2.620
	7	TO	TO	TO
Gated Katsura	6	**Singular 0.047**	GVW 0.226	facstd 0.233
	7	**Singular 0.283**	GVW 3.176	facstd 0.539
	8	**Singular 2.575**	TO	facstd 3.596
Gated dense	5	**Singular 0.025**	M4GB 0.171	facstd 0.198
	6	**Singular 0.552**	M4GB 4.736	facstd 0.755
	7	**Singular 42.670**	TO	facstd 48.820
Square affine	(8, 4)	**M4GB 0.532**	TO	facstd 0.601
	(10, 5)	**F4 0.920**	TO	facstd 2.141
#systems solved		19	9	16
#fastest (ties counted)		16	3	0

Table 2
Fastest completed method of each class on the generated systems, with a 60-second limit; the fastest overall is in bold, and TO means that every method of the class timed out. All times are in seconds.

Factor branching pays off when the branches are far easier than the system. Many branches are affordable when each is cheap: positive affine $(12, 10, 6)$ has 833 hitting sets, and factor branching solves it 22 times faster than `facstd`. One branch as hard as the whole system removes the gain, as on gated Cyclic-7, where every method times out.

5.3. Size-graded families

The following families inherit their factors from the problem, and we grow each until every method times out.

- The reduced cyclic n-roots systems `Singular.rcyclic_n`, a variant of the cyclic n-roots problem (Björck and Fröberg, 1991), for even n.
- Nash equilibria of n-player games with two pure strategies per player (Sturmfels, 2002). Each player i either plays a pure strategy, $p_i(p_i - 1) = 0$, or is indifferent, so its payoff difference vanishes. The payoffs are random integers in $[-9, 9]$ (generic) or in $\{-1, 0, 1\}$ (degenerate).
- Critical points of a random quadratic program f on every face of the box $[0, 1]^n$, given by $x_i(x_i - 1)\,\partial_i f = 0$.
- Equilibria of a random Lotka–Volterra community on every species support (Hofbauer and Sigmund, 1998), given by $x_i\big(r_i + \sum_j a_{ij}x_j\big) = 0$ with $a_{ii} < 0$.

Each size is one seeded instance, and a method that times out is not run at larger sizes. The direct class adds Singular's `std` in degrevlex (Singular dp) and drops standalone F4.

System	n	Best factor branching	Best direct GB	Best decomposition
rcyclic	8	F4 0.019	**GVW 0.008**	facstd 0.425
	10	**F4 0.018**	M4GB 0.041	TO
	12	**F4 0.021**	M4GB 0.198	TO
	14	**F4 0.029**	Singular dp 0.676	TO
	16	**Singular 0.062**	Singular dp 7.154	TO
	18	**Singular 0.093**	M2 34.60	TO
	20	**Singular 0.132**	TO	TO
Nash generic	3	F5 0.044	**F5 0.006**	facstd 0.197
	4	**GVW 0.136**	Singular dp 0.262	facstd 0.262
	5	**Singular 1.382**	TO	TO
	6	TO	TO	TO
Nash degenerate	3	M4GB 0.043	**GVW 0.005**	facstd 0.179
	4	**F5 0.086**	Singular lex 0.215	facstd 0.203
	5	**Singular 0.714**	TO	TO
	6	TO	TO	TO
Box-QP	5	F5 0.700	**Singular dp 0.216**	facstd 0.297
	6	M4GB 2.174	**F5 0.033**	primdec 0.939
	7	**F5 6.494**	TO	facstd 12.63
	8	**F5 20.42**	TO	TO
	9	TO	TO	TO
Lotka–Volterra	6	M4GB 0.191	**F5 0.006**	facstd 0.216
	8	F4 0.809	Singular dp 0.701	**facstd 0.464**
	10	**GVW 3.002**	TO	facstd 6.432
	12	**F5 13.15**	TO	TO
	14	TO	TO	TO
#systems solved		21	14	11
#fastest (ties counted)		14	6	1

Table 3
Fastest completed method of each class on the size-graded families, with a 60-second limit; the fastest overall is in bold. All times are in seconds.

Factor branching completes 21 of the 25 systems and is fastest on 14; the direct and decomposition classes complete 14 and 11 (Table 3). The baselines win only on systems that they finish in under 0.5 seconds. On the larger systems

of each family, factor branching is fastest, and its lead grows with n, to 372 times on rcyclic-18. Only factor branching completes rcyclic-20, the Nash games with $n = 5$, box-QP with $n = 8$, and Lotka–Volterra with $n = 12$; rcyclic-20 refines into three terminal branches, and box-QP with $n = 8$ into 6534 cheap ones.

Factor branching can also use a decomposition method as its branch solver. Table 4 compares `facstd` as the factor branching (FB) backend with `facstd` alone, and each new algorithm with the Singular command that has the same output: Algorithm 5 (Primary) with primdec, and prime factor branching (Prime, Theorem 5) with `minAssGTZ`. Here $\mathcal{B}_{\min}$ computes a factorizing Gröbner basis (Gräbe, 1995), returns each linear piece as a prime, and decomposes the other pieces with `minAssGTZ` (Gianni et al., 1988); only positive-dimensional primes prune later branches. Algorithm 5 uses the factorizing Gröbner basis in Algorithm 4, $\mathcal{B}_{\min}$ for the minimal primes, and the separators and zero-dimensional shortcut of Subsection 4.2, so it runs primdec only when an embedded component exists. In the last two pairs, refinement uses lex order and every Gröbner basis degrevlex, and the methods run in one Python process that drives one Singular process and exclude the input conversion, so their times are not comparable with the `facstd` pair or with Table 3. A method that timed out at a smaller size of a family is listed as TO, and sizes at which every method times out are omitted.

		`facstd`		primdec		`minAssGTZ`	
System	n	FB	alone	Primary	alone	Prime	alone
rcyclic	8	**0.038**	0.425	TO	TO	**0.081**	10.64
	10	**0.040**	TO	TO	TO	**0.110**	TO
	12	**0.043**	TO	TO	TO	**1.472**	TO
	14	**0.053**	TO	TO	TO	**21.22**	TO
	16	**0.079**	TO	TO	TO	TO	TO
	18	**0.162**	TO	TO	TO	TO	TO
	20	**0.458**	TO	TO	TO	TO	TO
Nash generic	3	**0.138**	0.197	0.065	**0.063**	**0.063**	0.073
	4	0.449	**0.262**	TO	TO	**0.121**	TO
	5	**13.48**	TO	TO	TO	TO	TO
Nash degenerate	3	**0.136**	0.179	0.087	**0.074**	**0.060**	0.069
	4	0.263	**0.203**	6.117	**5.004**	**0.088**	TO
	5	**1.634**	TO	TO	TO	TO	TO
Box-QP	5	1.928	**0.297**	**0.715**	16.20	**0.166**	17.00
	6	7.528	**1.259**	**0.423**	3.172	**0.446**	1.564
	7	22.77	**12.63**	TO	TO	**1.564**	TO
	8	TO	TO	TO	TO	**5.742**	TO
	9	TO	TO	TO	TO	**23.05**	TO
Lotka–Volterra	6	0.615	**0.216**	**0.101**	0.128	**0.096**	0.161
	8	3.012	**0.464**	**1.873**	TO	**0.254**	TO
	10	10.05	**6.432**	TO	TO	**1.059**	TO
	12	**41.47**	TO	TO	TO	**5.286**	TO
	14	TO	TO	TO	TO	**31.61**	TO

Table 4
Factor branching (FB) with `facstd`, Primary, and Prime against the Singular command with the same output, with a 60-second limit. The faster of each pair is in bold. All times are in seconds.

Factor branching with `facstd` completes every rcyclic system in under 0.5 seconds, the Nash games with $n = 5$, and Lotka–Volterra with $n = 12$; of these, `facstd` alone finishes only rcyclic-8. `facstd` alone wins on box-QP, the Nash games with $n = 4$, and the smaller Lotka–Volterra systems, where factor branching pays for starting Singular once per branch.

Prime finds the minimal primes of larger systems than `minAssGTZ` in every family, for example rcyclic-14 against rcyclic-8 and box-QP with $n = 9$ against $n = 6$. It stops when `minAssGTZ` cannot decompose a single terminal branch, such as the branch of the degenerate Nash game with $n = 5$ on which every player is indifferent.

Primary returns the same components as primdec wherever both finish. On these systems, Algorithm 4 stops after one round, so the gains come from avoiding primdec. The box-QP and Lotka–Volterra systems are zero-dimensional and hence have no embedded components, so primdec never runs: Primary is up to 23 times faster, on box-QP with $n = 5$, and alone finishes Lotka–Volterra with $n = 8$. With embedded components, it pays for both its own steps and primdec, as on the degenerate Nash game with $n = 4$. On rcyclic-8, it finds the three isolated components in 0.37 seconds, but primdec does not finish the embedded ones. Primary needs a Gröbner basis of the ideal, so it times out wherever `std` does, and also on the generic Nash game with $n = 4$, where `std` finishes.

5.4. Physical system

The physical system is an order-five approximation that constrains graviton couplings in asymptotic-safety models to remain consistent with quantum field theory (QFT) (Rafie-Zinedine, 2018; Vacca and Zanusso, 2010); solving it helps determine admissible fixed-point structures in quantum gravity (Vacca and Zanusso, 2010). It has 28 equations in 21 variables over $\mathbb{Q}(\pi)$, and the coupling κ_h must be nonzero.

Each run on the physical systems has a 120-second limit, or 300 seconds in Table 5, runs concurrently with the others, and reports its internal solver time. Factor branching and the Singular and Macaulay2 baselines saturate by κ_h; the standalone M4GB, F5, GVW, and F4 runs add the Rabinowitsch generator $1 - s\kappa_h$ (Rabinowitsch, 1930). Factor branching limits its saturation to five seconds, removes κ_h from every factor set, drops a branch once κ_h reduces to zero modulo its generators, and adds $1 - s\kappa_h$ to each terminal branch. Singular and Macaulay2 compute over $\mathbb{Q}(\pi)$, except Macaulay2's `decompose`, which cannot; the Rust programs take π as a variable. Saturations use Singular's `sat` or Macaulay2's `saturate`, primary decompositions use `primdecSY` (Shimoyama and Yokoyama, 1996), and Algorithm 5 runs its separators. Refinement and Algorithm 4 use lex order with the min-fill variable order, a greedy elimination order computed from the variable supports of the equations, and so do saturations and decompositions on the physical system.

All nine standalone baselines time out on the physical system, whereas factor branching with each of its six backends, those of Subsection 5.1 and `facstd`, refines the saturated system into 10 SAT terminal branches and solves them in 0.87 to 0.95 seconds, about 0.83 seconds of which go to the saturation.

Keeping κ_h nonzero changes only the SAT test of Algorithm 4: a minimal hitting set H counts as SAT when $V(H) \not\subseteq V(\kappa_h)$. The proof of Lemma 3 then shows that Q and the cofactors of h have no common zero outside $V(\kappa_h)$, so the Nullstellensatz gives $\kappa_h^s h^{a_h} \in \langle Q \rangle$ for some $s \geq 0$. Each round therefore keeps $\langle Q \rangle : \kappa_h^\infty$ instead of $\langle Q \rangle$, and the proof of Theorem 6 still applies. Algorithm 5 then decomposes $\langle Q \rangle : \kappa_h^\infty$, whose minimal primes are those of $\langle Q \rangle$ that do not contain κ_h.

Algorithm 5 decomposes the physical system in 271 seconds (Table 5). Ideal reduction turns the 28 equations into 18 generators in three rounds and 0.6 seconds, and prime factor branching finds the six minimal primes in 0.1 seconds. The separators give the isolated components in 1.6 seconds, but these do not intersect to the ideal, so `primdecSY` decomposes the saturation of the reduced system, with 56 generators, in most of the remaining time. On the original equations with a Rabinowitsch generator, `primdecSY` does not finish within 30 minutes. The six isolated components form two families with dimensions 13, 12, and 8. The components of one family are prime, and those of the other are obtained from their radicals by squaring one generator, a variable, which doubles the degree. Each family also has two embedded components, of dimension 10 inside its 12-dimensional component and of dimension 8 inside its 13-dimensional one. A Gröbner basis of the saturation confirms that the ten components intersect to $\langle P \rangle : \kappa_h^\infty$.

For each $\alpha \in \{1, 1/2, 1/4, 2, 4\}$, we also derive two systems from the Einstein–Hilbert truncation of gravity coupled to fermions and scalars, each with 31 equations once the Rabinowitsch equation of the coupling `KappaRefFinal` is replaced by saturation. The graviton-only system sets $d_1 = d_2 = d_3 = 0$ (29 or 30 variables), and the fermion-only system sets every variable whose name begins with Dp or p to zero (15 variables). On these systems, saturations and decompositions use degrevlex with the variables in name order, which is faster there. Table 5 runs the pairs of Table 4 on these systems and on the physical system. Every method keeps κ_h, or `KappaRefFinal`, nonzero: the commands

alone saturate first, factor branching as described above, Primary by Algorithm 4 and a saturation of its output, and Prime by dropping the minimal primes that contain the coupling.

		facstd		primdec		minAssGTZ	
System	α	FB	alone	Primary	alone	Prime	alone
Physical		**0.954**	TO	**271.3**	TO	**0.962**	TO
Graviton only	all	TO	TO	TO	TO	TO	TO
Fermion only	1	0.035	**0.001**	0.059	**0.007**	0.267	**0.007**
	1/2	0.028	**0.001**	0.060	**0.007**	0.229	**0.007**
	1/4	0.037	**0.001**	0.060	**0.006**	0.231	**0.007**
	2	0.028	$<$ **0.001**	0.060	**0.007**	0.231	**0.007**
	4	0.031	**0.001**	0.060	**0.006**	0.240	**0.007**

Table 5
The pairs of Table 4 on the physical systems, with a 300-second limit. The faster of each pair is in bold. Times are internal solver times in seconds.

On the physical system, every command alone times out, whereas factor branching with `facstd` and Prime finish in under a second and Primary in 271 seconds. No method finishes a graviton-only system. Each fermion-only system refines into a single branch and has a single prime and primary component, which every method finds within 0.3 seconds; the commands alone win every pair there.

6. Conclusion

We introduced a solver-agnostic framework that decomposes polynomial systems into smaller branches by enumerating minimal hitting sets of their factor sets.

We proved that refinement terminates and preserves the full solution set, and that the variants return the minimal primes, a generating set of the ideal, and a minimal primary decomposition.

On the baseline benchmarks, factor branching completed 19 of 20 systems, compared with 9 for direct Gröbner-basis methods and 16 for decomposition methods, and on the size-graded families 21 of 25, with a lead that grew with the system size. On the QFT system, where one coupling must be nonzero, imposing this by saturation lets factor branching complete with all six backends in under one second, while all nine standalone baselines time out. Primary factor branching decomposes the same saturated system in under five minutes.

Future work will find additional ways to break down polynomial systems into smaller branches which apply to a wider range of systems.

CRediT authorship contribution statement

Ilay Menahem: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Writing – original draft, Visualization. **Michael Shalyt:** Writing – review & editing. **Ido Kaminer:** Writing – review & editing, Visualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The code and data associated with this work are publicly available at `https://github.com/IlayMenahem/soothing`.

Acknowledgments

This work did not receive any dedicated funding, and the authors have no one to acknowledge beyond those credited as authors above.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used generative AI tools to assist with drafting and editing portions of the manuscript text. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

Atiyah, M.F., Macdonald, I.G., 1969. Introduction to Commutative Algebra. Addison-Wesley, Reading, MA.

Berge, C., 1989. Hypergraphs: Combinatorics of Finite Sets. North-Holland, Amsterdam.

Björck, G., Fröberg, R., 1991. A faster way to count the solutions of inhomogeneous systems of algebraic equations, with applications to cyclic *n*-roots. Journal of Symbolic Computation 12, 329–336. doi:10.1016/S0747-7171(08)80153-8.

Böge, W., Gebauer, R., Kredel, H., 1986. Some examples for solving systems of algebraic equations by calculating Gröbner bases. Journal of Symbolic Computation 2, 83–98. doi:10.1016/S0747-7171(86)80014-1.

Buchberger, B., 2006. Bruno Buchberger's PhD thesis 1965: An algorithm for finding the basis elements of the residue class ring of a zero dimensional polynomial ideal. Journal of Symbolic Computation 41, 475–511. doi:10.1016/j.jsc.2005.09.007.

Cox, D.A., Little, J., O'Shea, D., 2015. Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra. 4 ed., Springer, Cham. doi:10.1007/978-3-319-16721-3.

Decker, W., Greuel, G.M., Pfister, G., Schönemann, H., 2025. Singular 4-4-1: A computer algebra system for polynomial computations. https://www.singular.uni-kl.de.

Dershowitz, N., Manna, Z., 1979. Proving termination with multiset orderings. Communications of the ACM 22, 465–476. doi:10.1145/359138.359142.

Eisenbud, D., Huneke, C., Vasconcelos, W., 1992. Direct methods for primary decomposition. Inventiones Mathematicae 110, 207–235. doi:10.1007/BF01231331.

Faugère, J.C., 1999. A new efficient algorithm for computing Gröbner bases (F4). Journal of Pure and Applied Algebra 139, 61–88. doi:10.1016/S0022-4049(99)00005-5.

Faugère, J.C., 2002. A new efficient algorithm for computing Gröbner bases without reduction to zero (F5), in: Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation, ACM. pp. 75–83. doi:10.1145/780506.780516.

Gao, S., Volny, IV, F., Wang, M., 2016. A new framework for computing Gröbner bases. Mathematics of Computation 85, 449–465. doi:10.1090/MCOM/2969.

Gianni, P., Trager, B., Zacharias, G., 1988. Gröbner bases and primary decomposition of polynomial ideals. Journal of Symbolic Computation 6, 149–167. doi:10.1016/S0747-7171(88)80040-3.

Gräbe, H.G., 1995. On factorized Gröbner bases, in: Fleischer, J., Grabmeier, J., Hehl, F.W., Küchlin, W. (Eds.), Computer Algebra in Science and Engineering. World Scientific, Singapore, pp. 77–89. URL: https://www.informatik.uni-leipzig.de/~graebe/ComputerAlgebra/Publications/fgb.pdf.

Grayson, D.R., Stillman, M.E., 2026. Macaulay2, a software system for research in algebraic geometry. https://macaulay2.com. Version 1.26.06.

Hofbauer, J., Sigmund, K., 1998. Evolutionary Games and Population Dynamics. Cambridge University Press, Cambridge.

Kera, H., Ishihara, Y., Kambe, Y., Vaccon, T., Yokoyama, K., 2024. Learning to compute Gröbner bases, in: Advances in Neural Information Processing Systems. URL: https://openreview.net/forum?id=ZRz7XlxBzQ.

Kreuzer, M., Robbiano, L., 2000. Computational Commutative Algebra 1. Springer, Berlin. doi:10.1007/978-3-540-70628-1.

Makarim, R.H., Stevens, M., 2017. M4GB: An efficient Gröbner-basis algorithm, in: Proceedings of the 2017 ACM International Symposium on Symbolic and Algebraic Computation, ACM. pp. 293–300. doi:10.1145/3087604.3087638.

Mayr, E.W., Meyer, A.R., 1982. The complexity of the word problems for commutative semigroups and polynomial ideals. Advances in Mathematics 46, 305–329. doi:10.1016/0001-8708(82)90048-2.

Peifer, D., Stillman, M., Halpern-Leistner, D., 2020. Learning selection strategies in Buchberger's algorithm, in: Proceedings of the 37th International Conference on Machine Learning, PMLR. pp. 7575–7585. URL: https://proceedings.mlr.press/v119/peifer20a.html.

Rabinowitsch, J.L., 1930. Zum Hilbertschen Nullstellensatz. Mathematische Annalen 102, 520. doi:10.1007/BF01782361.

Rafie-Zinedine, S., 2018. Simplifying Quantum Gravity Calculations. Master's thesis. Lund University. ArXiv:1808.06086.

Ruijl, B., 2026. Symbolica: Modern computer algebra. https://symbolica.io. Version 3.

Shimoyama, T., Yokoyama, K., 1996. Localization and primary decomposition of polynomial ideals. Journal of Symbolic Computation 22, 247–277. doi:10.1006/jsco.1996.0052.

Sommese, A.J., Verschelde, J., Wampler, C.W., 2001. Numerical decomposition of the solution sets of polynomial systems into irreducible components. SIAM Journal on Numerical Analysis 38, 2022–2046. doi:10.1137/S0036142900372549.

Sommese, A.J., Wampler, II, C.W., 2005. The Numerical Solution of Systems of Polynomials Arising in Engineering and Science. World Scientific, Singapore. doi:10.1142/5763.

Sturmfels, B., 2002. Solving Systems of Polynomial Equations. volume 97 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, Providence, RI. doi:10.1090/cbms/097.

Vacca, G.P., Zanusso, O., 2010. Asymptotic safety in einstein gravity and scalar-fermion matter. Physical Review Letters 105, 231601. doi:10.1103/PhysRevLett.105.231601.